

3 Разработка программы на языке C в среде Geany

3.1 Описание алгоритмов

В рамках данной работы для упорядочивания элементов числового массива был выбран алгоритм сортировки слиянием (Merge Sort). Данный метод относится к классу устойчивых алгоритмов сортировки и базируется на фундаментальной алгоритмической парадигме «разделяй и властвуй» (divide and conquer). Суть данного подхода заключается в декомпозиции одной сложной задачи на несколько более мелких и простых подзадач, решение которых затем объединяется для получения итогового результата. В отличие от простых алгоритмов (таких как сортировка пузырьком или вставками), сортировка слиянием гарантирует стабильно высокую эффективность даже в худших сценариях распределения входных данных.

Механизм работы алгоритма состоит из трех последовательных этапов, которые выполняются рекурсивно:

Разделение (Divide): исходный неотсортированный массив данных делится ровно пополам на два подмассива примерно одинакового размера. Для этого вычисляется индекс середины отрезка.

Покорение (Conquer): каждая из полученных половин рекурсивно подвергается процедуре дальнейшего деления до тех пор, пока длина подмассивов не станет равной единице. Массив, состоящий из одного элемента, априори считается отсортированным.

Слияние (Combine): происходит последовательное объединение (слияние) двух отсортированных подмассивов меньшей длины в один упорядоченный массив большей длины. Этот процесс продолжается снизу вверх, пока все элементы не соберутся в единый результирующий массив.

Главным вычислительным узлом алгоритма является именно процедура слияния (merge). На этом этапе программа сравнивает первые элементы двух

подмассивов и наименьший из них копирует во временный массив. Индекс в подмассиве, из которого был взят элемент, сдвигается вправо, после чего сравнение повторяется. Процесс продолжается до тех пор, пока один из подмассивов не исчерпает себя, после чего остаток второго подмассива полностью дописывается в конец. По завершении слияния данные из временного массива копируются обратно в исходный.

Вычислительная сложность алгоритма сортировки слиянием во всех случаях (в лучшем, среднем и худшем) составляет $O(n \log n)$, где n — количество элементов в массиве. Такая стабильность достигается благодаря тому, что дерево рекурсивного деления всегда имеет глубину $\log n$, а на каждом уровне дерева выполняется ровно n операций сравнения и перестановки при слиянии подмассивов. Единственным относительным недостатком алгоритма является необходимость выделения дополнительной памяти объемом $O(n)$ для работы временных массивов на этапе слияния, что компенсируется высокой скоростью и предсказуемостью времени работы программы.

3.2 Создание проекта в IDE Geany

Чтобы создать проект нужно нажать на символ листка в левом верхнем углу и выбрать нужный шаблон. Изображение окна проекта представлено на рисунке 16.

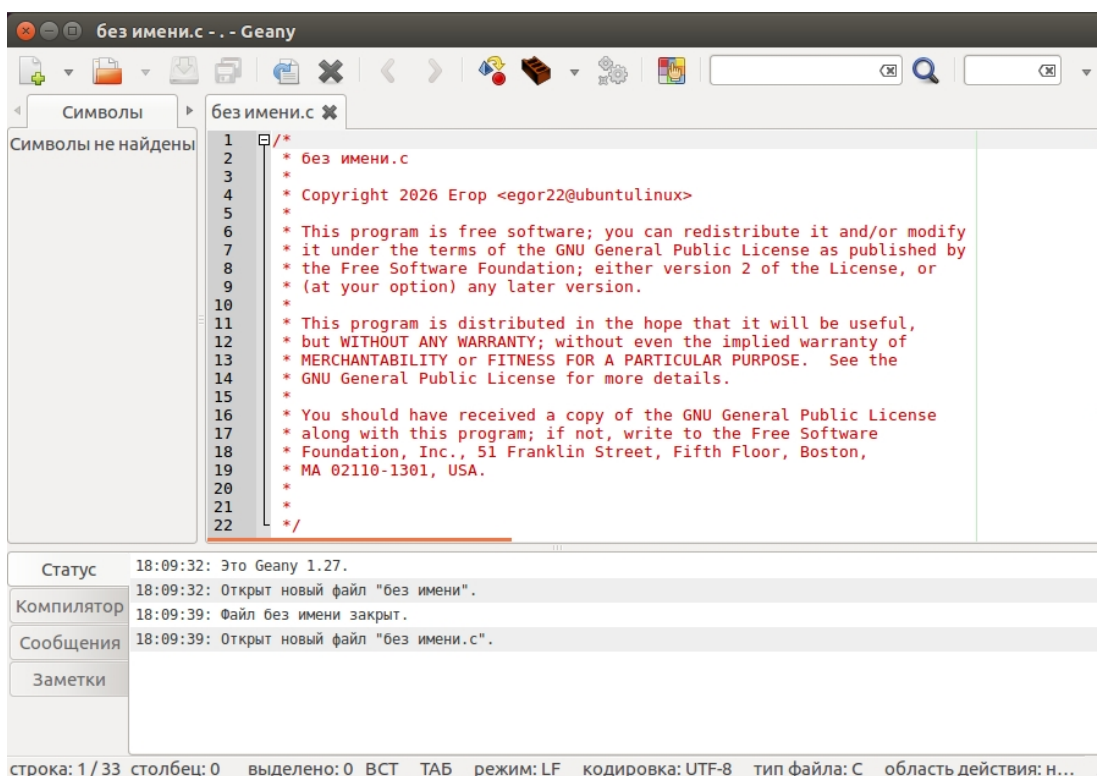


Рисунок 16 – Созданный файл

Затем нужно произвести сохранение файла в папку проекта и ввести название файла. Изображение представлено на рисунке 17.

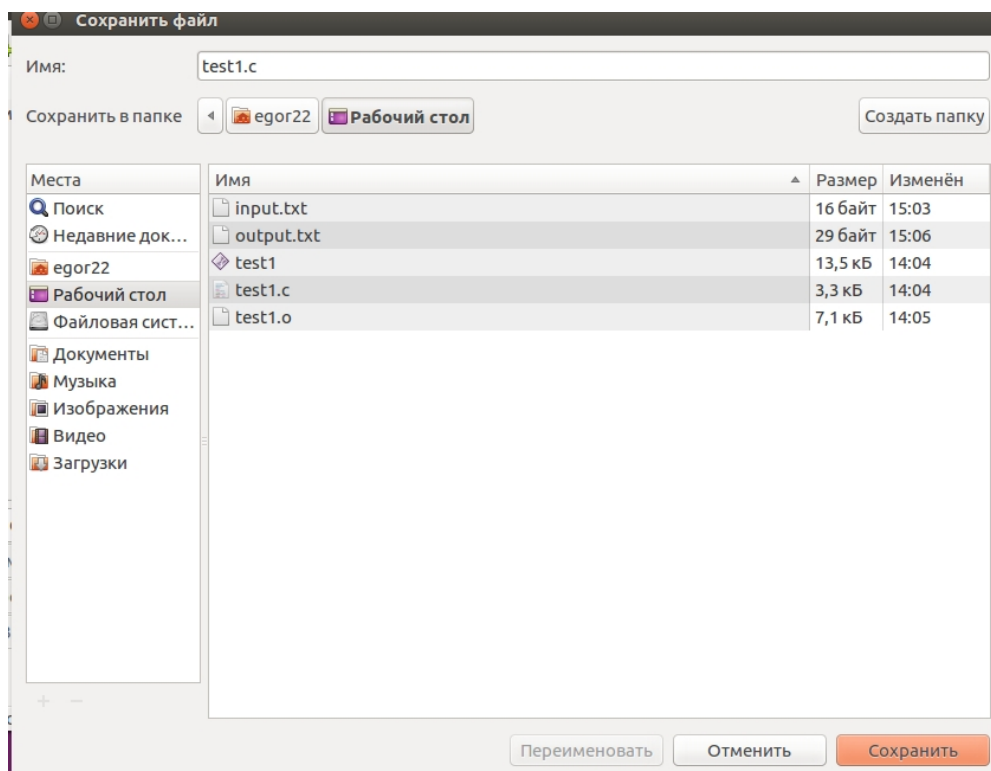


Рисунок 17 – Окно сохранения файла

3.3 Описание программы

Разработанное программное обеспечение представляет собой консольное приложение на языке программирования Си (C), предназначенное для автоматизации процесса упорядочивания числовых массивов методом слияния. Программа спроектирована с использованием принципов процедурного программирования, что позволило разделить общую вычислительную задачу на логически обособленные функциональные блоки. Архитектурно приложение включает в себя модули ввода-вывода информации, подсистему генерации псевдослучайных данных, процедуры обработки ошибок и непосредственно сам вычислительный модуль сортировки.

Взаимодействие пользователя с программой организовано посредством интерактивного текстового меню, обеспечивающего гибкость выбора источника данных. Программа поддерживает три режима работы, включая ручной ввод элементов числовой последовательности с клавиатуры, автоматическое считывание готового набора данных из внешнего текстового файла `input.txt` и генерацию массива фиксированной длины случайными целыми числами в диапазоне от 0 до 99. Вне зависимости от выбранного режима ввода, результирующий отсортированный массив выводится на экран терминала, а также автоматически экспортируется в выходной текстовый файл `output.txt`.

Структурно исходный код приложения распределен между четырьмя взаимосвязанными функциями, где центральное место занимает управляющая функция `main`. Она отвечает за инициализацию поддержки локализации (русского языка) с помощью `setlocale`, запуск генератора случайных чисел `srand`, обработку пользовательского выбора, координацию работы с файловыми потоками `FILE*`, а также последовательный вызов процедур вывода и сортировки. Процесс непосредственного декомпозиции массива

выполняет рекурсивная функция `mergeSort`, которая принимает границы отрезка, вычисляет индекс центрального элемента и разделяет задачу пополам до достижения базового условия выхода из рекурсии. Напрямую за упорядоченное объединение отвечает функция `merge`, осуществляющая динамическое выделение оперативной памяти под временные подмассивы `L` и `R` с помощью `malloc` и последующую её очистку через `free`. Визуализация текущего состояния числовой последовательности возложена на служебную функцию `printArray`, последовательно выводящую элементы в окно терминала.

Важной особенностью архитектуры программы является развитая система обработки исключительных ситуаций. Приложение осуществляет жесткий контроль на каждом этапе работы, проверяя успешность открытия файлов ввода-вывода, контролируя тип вводимых символов через возвращаемые значения функции `scanf`, блокируя переполнение статического массива при ограничении до 100 элементов, а также полностью пресекая попытки задания некорректной или отрицательной длины массива. Такой подход гарантирует высокую отказоустойчивость и стабильность работы приложения в гостевой операционной системе Linux Ubuntu.

3.4 Функциональное тестирование

В рамках функционального тестирования разработанного программного обеспечения ключевое внимание было уделено проверке устойчивости приложения к некорректным действиям пользователя. Целью испытаний являлась проверка корректной обработки исключительных ситуаций, таких как нечисловых символов, ввод отрицательных чисел, выбор несуществующих пунктов меню, а также задание размера массива вне допустимого диапазона (менее 1 или более 100 элементов). На рисунке 18 представлен результат работы программы, при вводе пользователем отрицательного числа.

```
Терминал
Сортировка слиянием:
1. Ввести данные вручную
2. Считать данные из файла (input.txt)
3. Сгенерировать массив случайно
1
Введите количество элементов (от 1 до 100): 3
Введите 3 элементов через пробел: 8 -5 3

Исходный массив: 8 -5 3
Отсортированный массив: -5 3 8

Результат сохранен в output.txt

-----
(program exited with code: 0)
Press return to continue
```

Рисунок 18 – Сортировка с отрицательным числом

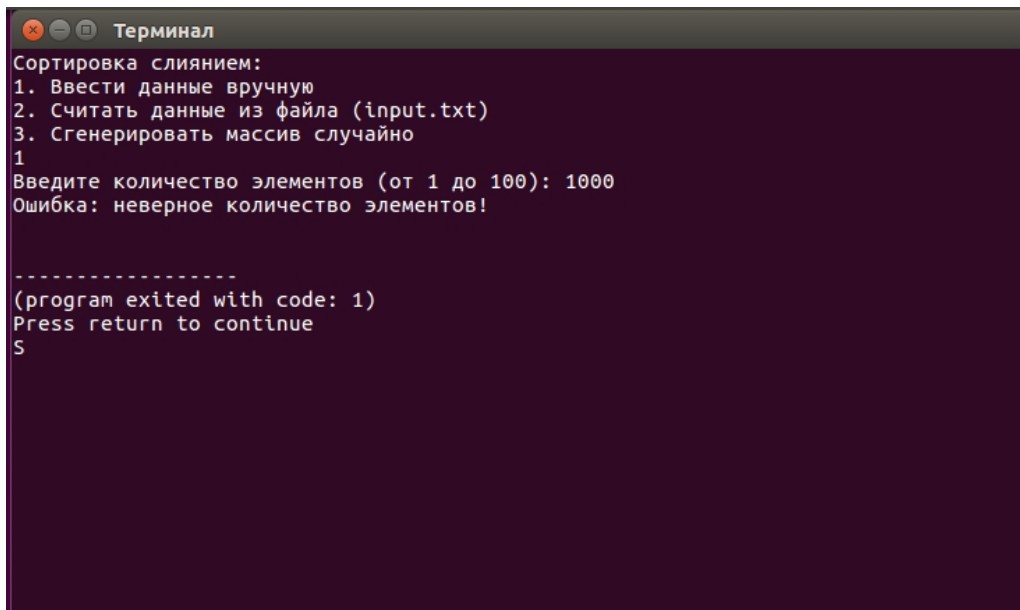
На рисунке 19 представлен результат работы программы, при вводе несуществующего пункта меню.

```
Терминал
Сортировка слиянием:
1. Ввести данные вручную
2. Считать данные из файла (input.txt)
3. Сгенерировать массив случайно
4
Ошибка: такого варианта нет в меню.

-----
(program exited with code: 1)
Press return to continue
```

Рисунок 19 – Выбор несуществующего пункта меню

На рисунке 20 представлен результат работы программы, при вводе недопустимого количества элементов массива.

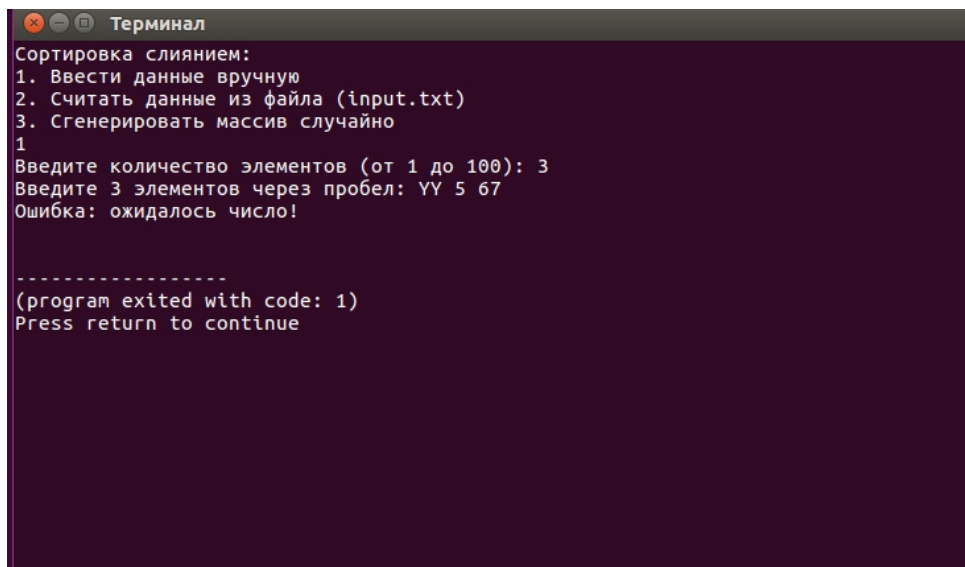


```
Терминал
Сортировка слиянием:
1. Ввести данные вручную
2. Считать данные из файла (input.txt)
3. Сгенерировать массив случайно
1
Введите количество элементов (от 1 до 100): 1000
Ошибка: неверное количество элементов!

-----
(program exited with code: 1)
Press return to continue
S
```

Рисунок 20 – Ввод недопустимого количества элементов массива

На рисунке 21 представлен результат работы программы, при вводе нечислового символа.



```
Терминал
Сортировка слиянием:
1. Ввести данные вручную
2. Считать данные из файла (input.txt)
3. Сгенерировать массив случайно
1
Введите количество элементов (от 1 до 100): 3
Введите 3 элементов через пробел: YY 5 67
Ошибка: ожидалось число!

-----
(program exited with code: 1)
Press return to continue
```

Рисунок 21 – Ввод нечислового символа

Заключение

В ходе выполнения практической работы была изучена и установлена операционная система Ubuntu на виртуальную машину VirtualBox, установлена интегрированная среда разработки Geany в ОС Ubuntu, были изучены основные инструменты среды и разработана программа на языке Си, реализующая алгоритм сортировки слиянием. Результаты испытаний показали, что программа работает согласно формулировке задачи.

Список используемой литературы

1. Полное руководство по всем видам тестирования. Режим доступа: <https://habr.com/ru/companies/otus/articles/923480/> (Дата обращения: 22.06.2026)
 2. Б.Керниган, Д.Ритчи. Язык программирования С. - Питер, 2015. - 304с (Дата обращения: 20.06.2026)
 3. Колисниченко, Д. Н. Ubuntu Linux. Экспресс-курс. – СПб.: БХВ-Петербург, 2019. – 320 с. (Дата обращения: 20.06.2026)
 4. Официальное руководство пользователя Oracle VM VirtualBox (Oracle VM VirtualBox User Manual). Режим доступа: <https://www.virtualbox.org/manual/> (Дата обращения: 22.06.2026)
 5. Официальная документация интегрированной среды разработки Geany. Режим доступа: <https://www.geany.org/documentation/> (Дата обращения: 22.06.2026)
- Ссылка на GitHub: <https://github.com/egor228dev/Praktikavvv2>

Приложение А

Листинг программы

```
#include <stdio.h>
#include <stdlib.h>
#include <locale.h>
#include <time.h>

void merge(int arr[], int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - mid;

    int* L = (int*)malloc(n1 * sizeof(int));
    int* R = (int*)malloc(n2 * sizeof(int));

    for (int i = 0; i < n1; i++) L[i] = arr[left + i];
    for (int j = 0; j < n2; j++) R[j] = arr[mid + 1 + j];

    int i = 0, j = 0, k = left;

    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) {
            arr[k] = L[i];
            i++;
        }
        else {
            arr[k] = R[j];
            j++;
        }
        k++;
    }

    while (i < n1) {
        arr[k] = L[i];
        i++;
        k++;
    }

    while (j < n2) {
        arr[k] = R[j];
        j++;
        k++;
    }

    free(L);
    free(R);
}

void mergeSort(int arr[], int left, int right) {
    if (left >= right) {
        return;
    }

    int mid = left + (right - left) / 2;
    mergeSort(arr, left, mid);
    mergeSort(arr, mid + 1, right);
    merge(arr, left, mid, right);
}

void printArray(int arr[], int size) {
    for (int i = 0; i < size; i++) {
        printf("%d ", arr[i]);
    }
}
```

```

    }
    printf("\n");
}

int main() {
    setlocale(LC_ALL, "Russian");
    srand(time(NULL));

    int arr[100];
    int n = 0;
    int choice;

    printf("Сортировка слиянием:\n");
    printf("1. Ввести данные вручную\n");
    printf("2. Считать данные из файла (input.txt)\n");
    printf("3. Сгенерировать массив случайно\n");

    if (scanf("%d", &choice) != 1) {
        printf("Ошибка: нужно ввести число от 1 до 3!\n");
        return 1;
    }

    if (choice == 1) {
        printf("Введите количество элементов (от 1 до 100): ");
        if (scanf("%d", &n) != 1 || n <= 0 || n > 100) {
            printf("Ошибка: неверное количество элементов!\n");
            return 1;
        }

        printf("Введите %d элементов через пробел: ", n);
        for (int i = 0; i < n; i++) {
            if (scanf("%d", &arr[i]) != 1) {
                printf("Ошибка: ожидалось число!\n");
                return 1;
            }
        }
    }
    else if (choice == 2) {
        FILE* fileIn = fopen("input.txt", "r");
        if (fileIn == NULL) {
            printf("Ошибка: файл input.txt не найден!\n");
            return 1;
        }

        while (fscanf(fileIn, "%d", &arr[n]) == 1) {
            n++;
            if (n >= 100) break;
        }
        fclose(fileIn);

        if (n == 0) {
            printf("Ошибка: файл пуст или в нем нет чисел!\n");
            return 1;
        }
        printf("Успешно считано %d элементов из файла.\n", n);
    }
    else if (choice == 3) {
        printf("Введите желаемый размер массива (от 1 до 100): ");
        if (scanf("%d", &n) != 1 || n <= 0 || n > 100) {
            printf("Ошибка: неверное количество элементов!\n");
            return 1;
        }

        for (int i = 0; i < n; i++) {
            arr[i] = rand() % 100;
        }
    }
}

```

```

    }
    printf("Массив успешно сгенерирован.\n");
}
else {
    printf("Ошибка: такого варианта нет в меню.\n");
    return 1;
}

printf("\nИсходный массив: ");
printArray(arr, n);

mergeSort(arr, 0, n - 1);

printf("Отсортированный массив: ");
printArray(arr, n);

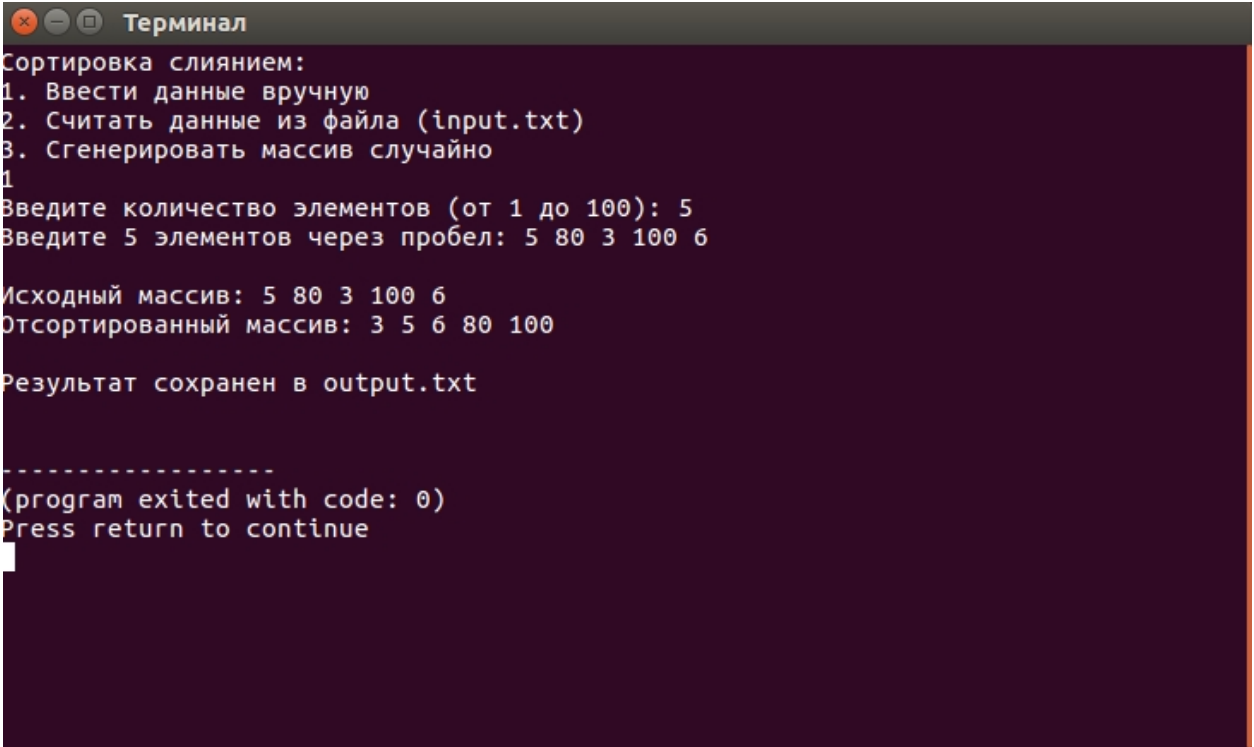
FILE* fileOut = fopen("output.txt", "w");
if (fileOut != NULL) {
    for (int i = 0; i < n; i++) {
        fprintf(fileOut, "%d ", arr[i]);
    }
    fclose(fileOut);
    printf("\nРезультат сохранен в output.txt\n");
}
else {
    printf("\nОшибка записи в файл.\n");
}

return 0;
}

```

Приложение Б

Результаты испытаний



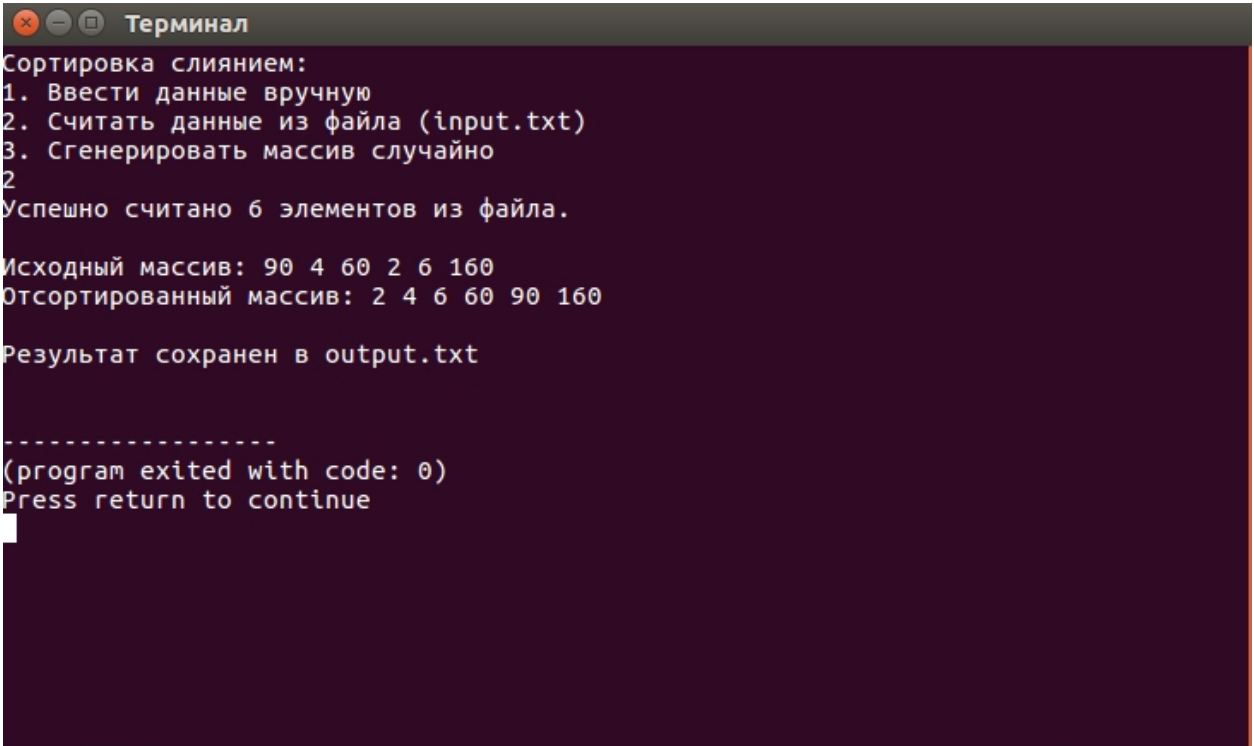
```
Терминал
Сортировка слиянием:
1. Ввести данные вручную
2. Считать данные из файла (input.txt)
3. Сгенерировать массив случайно
1
Введите количество элементов (от 1 до 100): 5
Введите 5 элементов через пробел: 5 80 3 100 6

Исходный массив: 5 80 3 100 6
Отсортированный массив: 3 5 6 80 100

Результат сохранен в output.txt

-----
(program exited with code: 0)
Press return to continue
```

Рисунок Б1 – Ручной ввод массива и его сортировка



```
Терминал
Сортировка слиянием:
1. Ввести данные вручную
2. Считать данные из файла (input.txt)
3. Сгенерировать массив случайно
2
Успешно считано 6 элементов из файла.

Исходный массив: 90 4 60 2 6 160
Отсортированный массив: 2 4 6 60 90 160

Результат сохранен в output.txt

-----
(program exited with code: 0)
Press return to continue
```

Рисунок Б2 – Чтение и сортировка массива из файла



Рисунок Б3 – Содержимое файла с заранее заданными числами

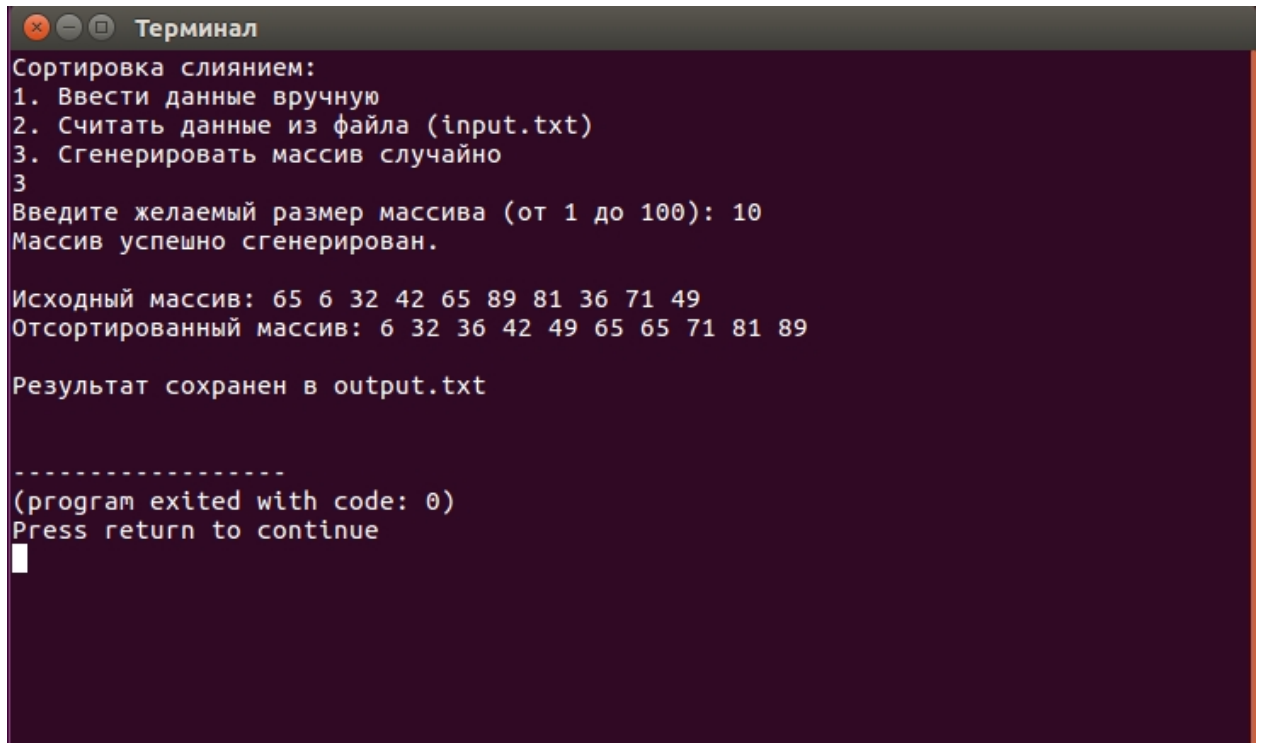


Рисунок Б4 – Случайно сгенерированный массив и его сортировка

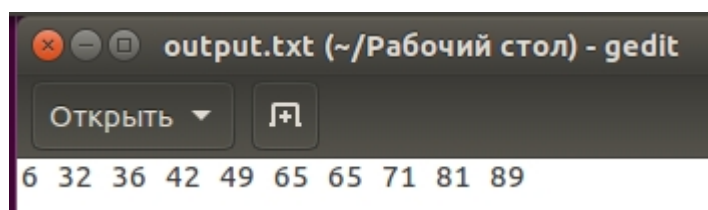
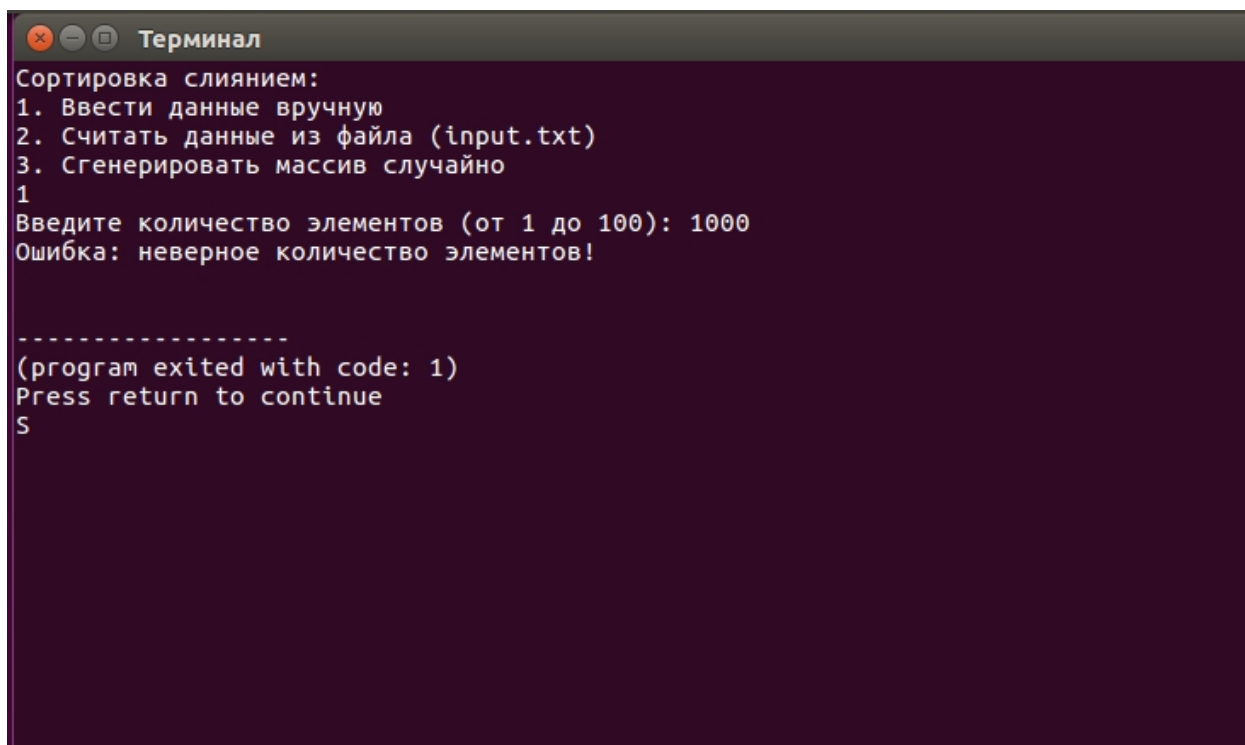


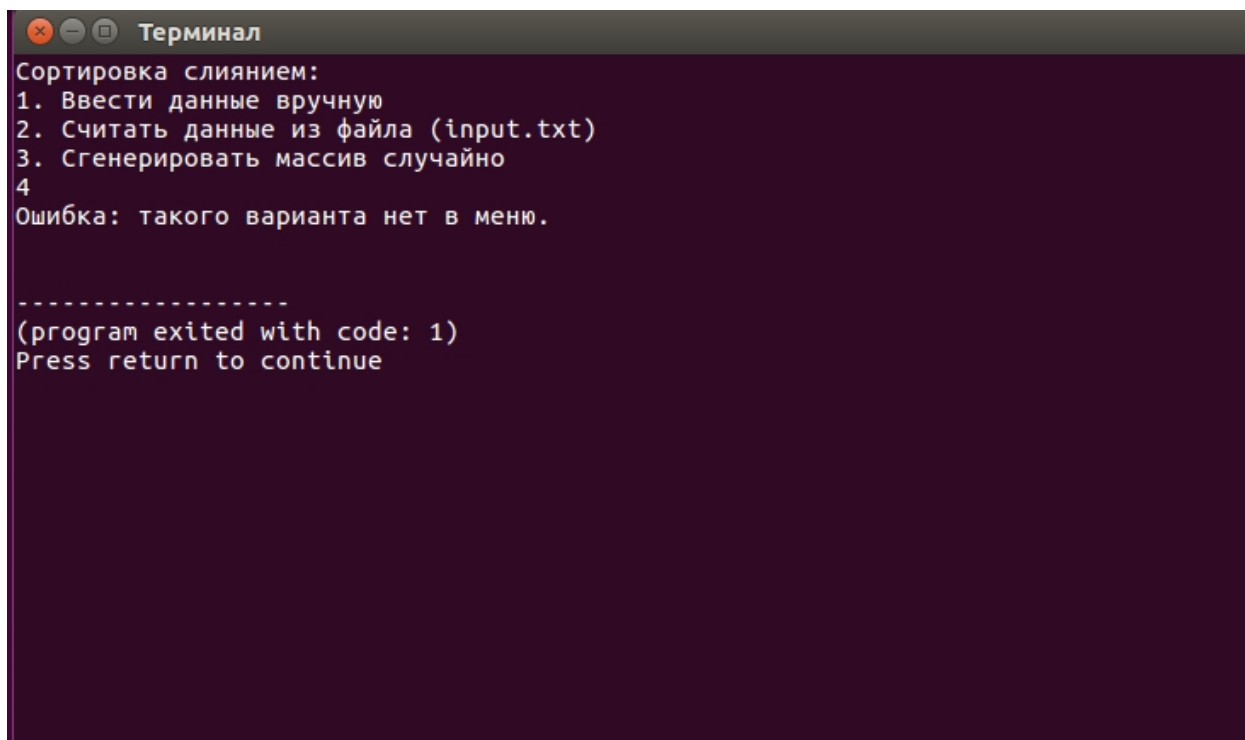
Рисунок Б5 – Успешная загрузка результата в файл



```
Терминал
Сортировка слиянием:
1. Ввести данные вручную
2. Считать данные из файла (input.txt)
3. Сгенерировать массив случайно
1
Введите количество элементов (от 1 до 100): 1000
Ошибка: неверное количество элементов!

-----
(program exited with code: 1)
Press return to continue
5
```

Рисунок Б6 – Ввод некорректного количества элементов массива



```
Терминал
Сортировка слиянием:
1. Ввести данные вручную
2. Считать данные из файла (input.txt)
3. Сгенерировать массив случайно
4
Ошибка: такого варианта нет в меню.

-----
(program exited with code: 1)
Press return to continue
```

Рисунок Б7 – Ввод некорректного номера операции